TYSD FIELD PROJECT | ACADEMIC YEAR 2026-27

AI Interview Preparation Platform

Basic Modules, Data Design, Schema Design, and Data Integrity & Constraints

Student Name	_____
Roll Number	_____
Class / Division	_____
Project Guide	_____
College	_____
Submission Date	_____

Submission: 4.1 Basic Modules | 4.2 Data Design | 4.2.1 Schema Design | 4.2.2 Data Integrity and Constraints

Executive Summary

The AI Interview Preparation Platform is a full-stack web application that helps users prepare for job interviews through authenticated accounts, resume analysis, role-based question generation, and guided mock interviews. React and Vite provide the user interface, while Node.js and Express implement the API. MongoDB Atlas is the permanent data store, and Mongoose defines collection schemas and validation rules.

Database purpose: MongoDB stores permanent user, resume, interview, and question records. Browser localStorage is used only as a temporary frontend cache and is not the authoritative data source.

Document Scope

- 4.1 Basic Modules - functional components currently implemented in the platform.
- 4.2 Data Design - database structure, collection ownership, and data flow.
- 4.2.1 Schema Design - field definitions for Users, Resumes, Interviews, and Questions.
- 4.2.2 Data Integrity and Constraints - validation, security, uniqueness, and ownership controls.

4.1 Basic Modules

No.	Module	Responsibility	Status
1	Registration and Login	Creates accounts, hashes passwords, verifies credentials, and issues JWTs.	Implemented
2	Resume Upload	Accepts authenticated PDF/DOCX uploads up to 5 MB using Multer.	Implemented
3	Resume Analysis	Extracts text, skills, education, experience, and keywords and stores the result.	Implemented
4	Resume Profile Editing	Allows users to correct target role, experience level, skills, education, experience, and keywords.	Implemented
5	Question Generation	Uses Gemini first and a permanent role-aware local fallback if the API is unavailable.	Implemented
6	Mock Interview	Displays saved questions one at a time with type, difficulty, source, and progress.	Implemented
7	Answer Evaluation	Will evaluate correctness, confidence, communication, and technical accuracy.	Planned
8	Analytics and History	Will show previous interviews, scores, progress charts, and suggestions.	Planned

Module Interaction

The modules execute in a controlled sequence: registration -> login -> resume upload -> resume analysis -> profile review/editing -> interview configuration -> question generation -> mock interview. Evaluation and analytics will extend this sequence in later modules.

4.2 Data Design

MongoDB Atlas was selected because the project stores nested resume-analysis data, evolving interview records, and user-specific documents. Mongoose provides application-level schemas, validation, indexes, references, defaults, and data-type enforcement.

Collection Relationship Model

Parent	Child	Cardinality	Reference
Users	Resumes	One-to-many	Resumes.userId -> Users._id
Users	Interviews	One-to-many	Interviews.userId -> Users._id
Resumes	Interviews	One-to-many	Interviews.resumeId -> Resumes._id
Interviews	Questions	One-to-many	Questions.interviewId -> Interviews._id

Ownership rule: Every protected read or update filters by the authenticated user ID. A user cannot retrieve or edit another user's resume or interview by changing a URL identifier.

4.2.1 Schema Design

Users Collection

Stores registered user identities. Password values contain bcrypt hashes and are excluded from normal model queries.

Field	Type	Required	Description / Constraint
_id	ObjectId	Yes	MongoDB document identifier
name	String	Yes	Trimmed; 2-80 characters
email	String	Yes	Unique, lowercase, and trimmed
password	String	Yes	bcrypt hash; select:false
createdAt	Date	Yes	Defaults to current date/time

Resumes Collection

Stores file metadata, the authenticated owner, interview preferences, extracted text, and editable structured analysis.

Field	Type	Required	Description / Constraint
_id	ObjectId	Yes	Resume identifier
userId	ObjectId ref User	Owner	Connects the resume to its user
fileName	String	Yes	Original uploaded filename
storedFileName	String	Yes	Unique safe server filename
filePath	String	Yes	Relative path under server/uploads
mimeType	String	Yes	PDF or DOCX MIME type
interviewPreferences.role	String	Defaulted	Editable target job role
interviewPreferences.experienceLevel	String	Defaulted	Fresher, 1-2, 3-5, or 5+ Years
interviewPreferences.difficulty	String	Defaulted	Beginner, Intermediate, or Advanced
analysis.skills	String[]	Default []	Recognized and user-edited skills
analysis.education	String[]	Default []	Education lines
analysis.experience	String[]	Default []	Experience lines
analysis.keywords	String[]	Default []	High-value resume terms
analysis.rawText	String	Yes	Complete extracted document text
uploadedAt	Date	Defaulted	Upload timestamp

Interviews Collection

Field	Type	Required	Description / Constraint
_id	ObjectId	Yes	Interview identifier
userId	ObjectId ref User	Yes	Indexed owner reference
resumeId	ObjectId ref Resume	Yes	Source resume reference
role	String	Yes	Trimmed; maximum 120 characters
type	String enum	Yes	HR, Technical, Behavioral, or Mixed
difficulty	String enum	Yes	Beginner, Intermediate, or Advanced
questionCount	Number	Yes	Minimum 5; maximum 15

Field	Type	Required	Description / Constraint
generationSource	String enum	Default local	gemini or local
generationModel	String/null	No	Successful Gemini model identifier
status	String enum	Default	not_started, in_progress, or completed
score	Number/null	No	Future score constrained to 0-100
feedback	String	Default empty	Future evaluation feedback
createdAt	Date	Defaulted	Creation timestamp

Questions Collection

Field	Type	Required	Description / Constraint
_id	ObjectId	Yes	Question identifier
interviewId	ObjectId ref Interview	Yes	Indexed parent interview
order	Number	Yes	Minimum 1; unique within interview
question	String	Yes	Trimmed question text
type	String enum	Yes	HR, Technical, or Behavioral
difficulty	String enum	Yes	Beginner, Intermediate, or Advanced
skill	String/null	No	Related skill or competency
expectedPoints	String[]	Default []	Reference points for future evaluation

4.2.2 Data Integrity and Constraints

Area	Constraint / Control	Integrity Benefit
Identity	Unique email index	Prevents duplicate registered accounts.
Credential security	bcrypt hashing	Plain-text passwords are never stored.
Authorization	JWT + ownership filter	Protects private routes and prevents cross-user access.
File integrity	Extension, MIME, and size validation	Only PDF/DOCX files up to 5 MB are accepted.
Resume editing	Array cleaning and item limits	Removes blanks/duplicates and limits field sizes.
Interview type	Mongoose enum	Rejects types outside HR, Technical, Behavioral, and Mixed.

Area	Constraint / Control	Integrity Benefit
Difficulty	Mongoose enum	Rejects levels outside Beginner, Intermediate, and Advanced.
Question count	min 5 / max 15	Prevents invalid or excessive generation requests.
Question order	Compound unique index	One order number per question within an interview.
Score	min 0 / max 100	Prevents invalid future evaluation scores.
AI response	Strict JSON schema + validation	Rejects malformed, duplicate, or wrong-type Gemini output.
Availability	Model failover + local fallback	External API failures do not crash question generation.

Validation Layers

- Frontend validation gives immediate feedback for required fields, file type, file size, and form ranges.
- Express controllers validate request values and authenticated ownership before database operations.
- Mongoose schemas enforce types, required values, defaults, enum lists, numeric ranges, and indexes.
- MongoDB Atlas provides persistent storage independent of browser refreshes or application restarts.

End-to-End Data Flow

Step	Application Action	Collection	Database Operation
1	Register / Login	Users	Create user; return JWT after verification
2	Upload Resume	Resumes	Store metadata and parsed analysis
3	Edit Profile	Resumes	Save corrected role and structured resume fields
4	Generate Interview	Interviews	Store configuration, source, model, and status
5	Generate Questions	Questions	Store ordered role-specific questions
6	Mock Interview	Interviews + Questions	Load only records owned by current user

Working Module Evidence

The following screenshots demonstrate the implemented frontend workflow. Screenshots are included as implementation evidence; database credentials and API keys are not displayed.

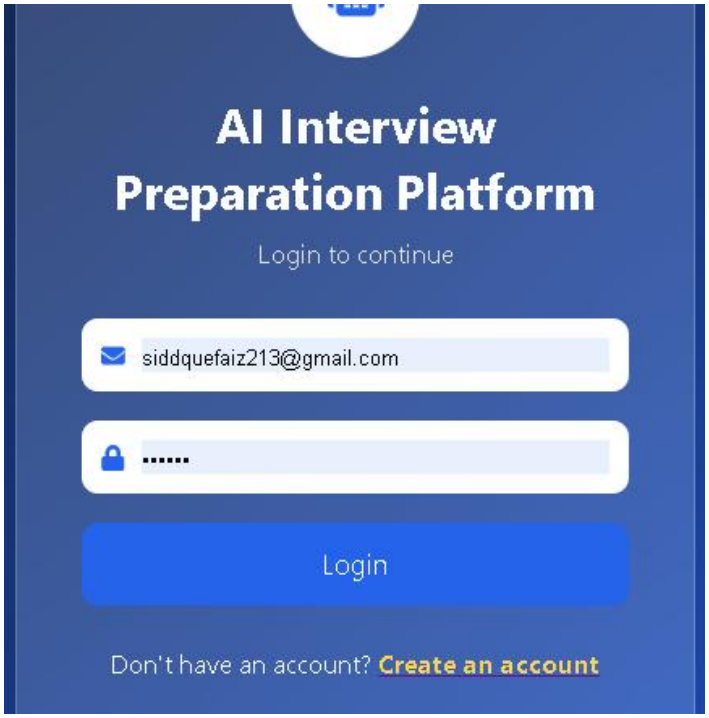


Figure 1. Authenticated entry point - Login interface

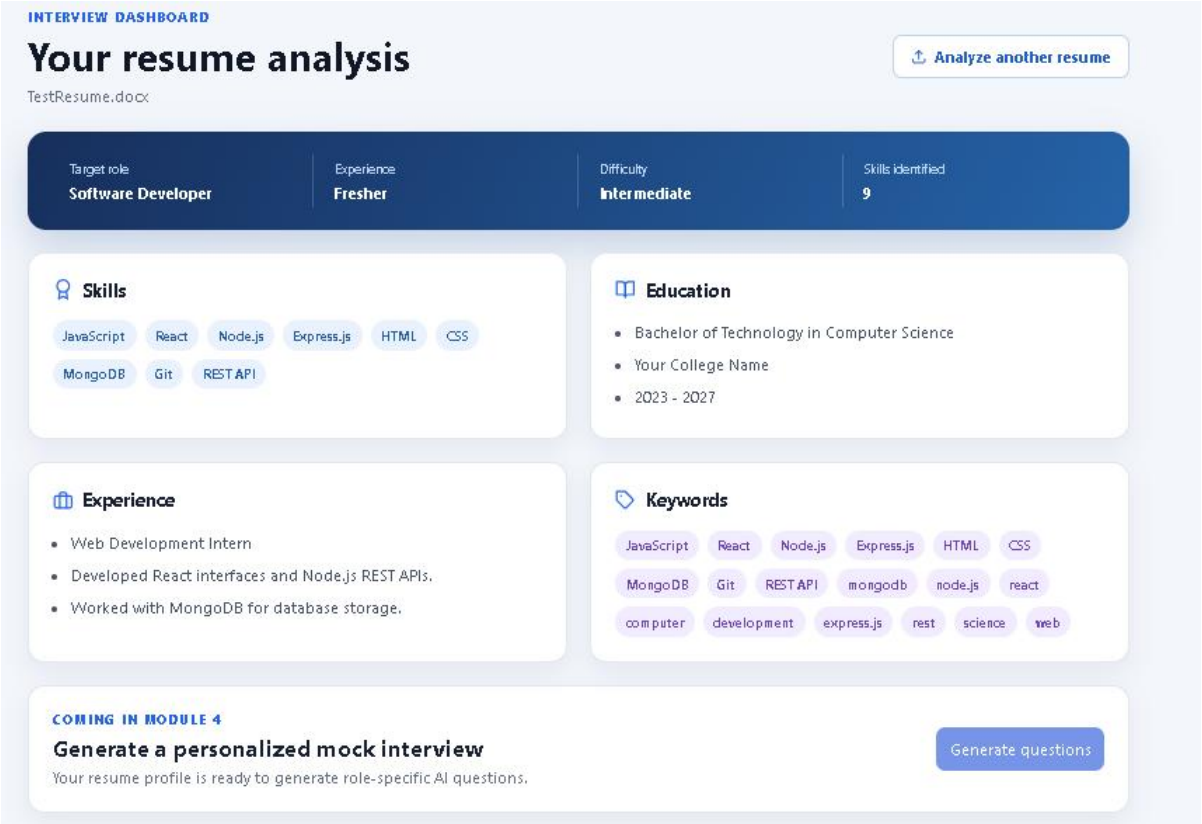


Figure 2. Resume Analysis Dashboard - extracted skills, education, experience, and keywords

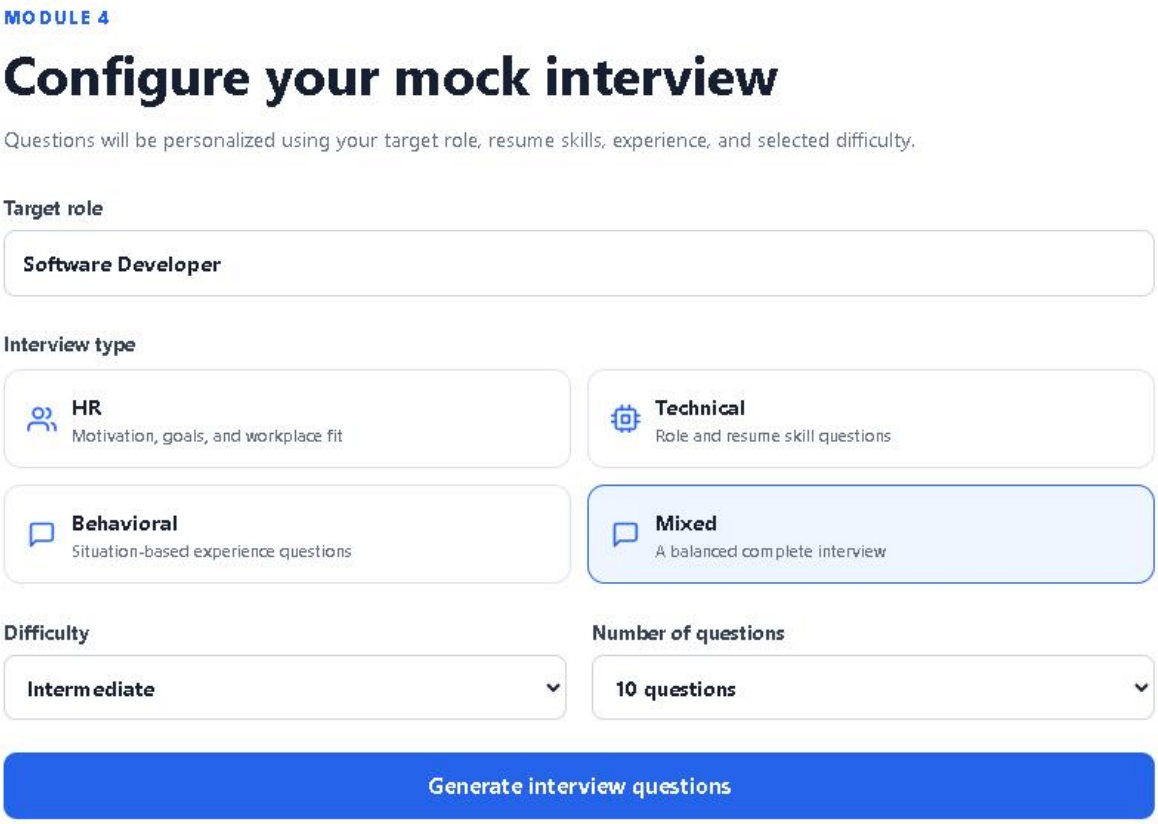


Figure 3. Mock Interview configuration - role, type, difficulty, and question count

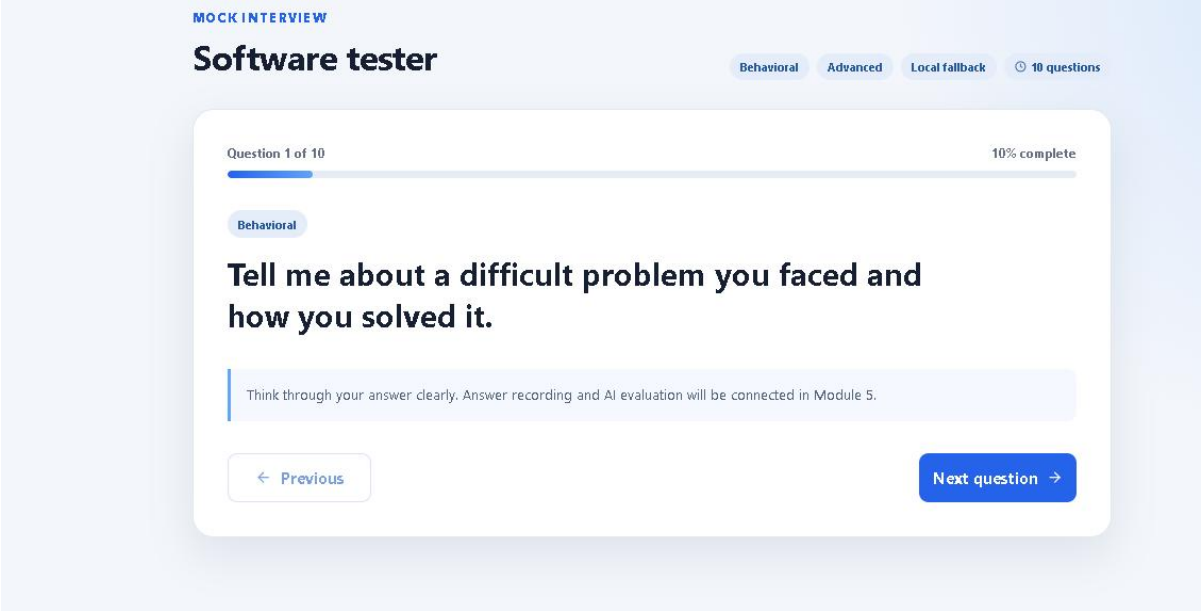


Figure 4. Mock Interview - question progress and visible generation source

Testing Summary

Test Area	Evidence	Result
Authentication	Unknown accounts rejected; registered user receives JWT	Pass
Resume upload	DOCX upload parsed and saved in MongoDB	Pass
Ownership	Latest resume/interview filtered by authenticated user	Pass
Question uniqueness	15 unique questions across types and difficulties	Pass
Difficulty variation	Distinct Beginner, Intermediate, and Advanced banks	Pass
Gemini integration	Flash-Lite generated five structured questions	Pass
Fallback	Local role-aware questions returned when Gemini unavailable	Pass
Frontend quality	ESLint and Vite production build completed	Pass

Conclusion

The current platform implements a modular and secure foundation for AI-assisted interview preparation. MongoDB Atlas provides durable, user-specific storage; Mongoose schemas document and enforce the data model; JWT authentication protects ownership; and the hybrid Gemini/local generator keeps the mock interview usable during external API failures. The schema is prepared for answer evaluation, performance reports, and interview history in subsequent modules.

Submission note: Before submission, replace the cover-page blanks with your student details. If your instructor requires MongoDB Atlas screenshots, add a redacted screenshot of each collection without exposing connection strings, passwords, JWTs, or API keys.